Python basics :

- Computer program = list of instructions
- Instruction in programming = statements
- Python is a simple programming language.
- Python was created by Guido Van Rossum in 1991.
- Extension for python file - .py
- In python Inditation is very important and used to indicate blocks of code.
- Python is a procedure oriented programming language.
- Semicolon is optional in python.
- To display text or value , use **print()** function
- Text must be written in quotes either double(**“**) or single(**‘**).
- By default , print() ends with a new line.
- If you want to print multiple messages in a same line then you can use **end =” “** parameter
  Ex., print(“hello!”, end=” “)
  print(“I’m Sayma”)
  Output : hello! I’m Sayma

- Don’t use quotes while printing numbers. ex.,print(3)
- Single line comment : #_____
- Multi line comment : “”” _____ _____ “”

Python variables:

- In python , there is no command for variable declaration.
- Variable is created when you assign value to it
- Ex., x = 5; x = “sayma”;
- If you want to mention data type of variable then you will write it as , x = str(3), y = int(3), z = float(3)
- We can also get data type using **type()** function
  Ex., x = 6;
  print(type(x)) ;

- Variable names are case sensitive .
  Ex., a = 4;
  A = 5;
  ‘A’ and ‘a’ are both different names of variable

- **Variable name rules:**
  1. Variable names must start with underscore and alphabet.
  2. Cannot contain special symbols.

3. It is case sensitive.
4. Cannot be keywords.
5. Cannot start with numbers.
6. Cannot contain blank space.

- We can assign multiple values to multiple variables
  Ex., x , y , z = “apple” , “banana” , “pig”
  print(x) # apple
  print(y) #banana
  print(z) #pig
- We can assign single value to multiple variables
  Ex., x = y = z = “orange”
  print(x) #orange
  print(y) #orange
  print(z) #orange
- If you want to print multiple variable’s output then you can use comma(,) like “print(x,y,z)” or + operator like “print(x+y+z)”. Here x , y , and z are variables .
- Global variable = outside the function
- Local variable = inside the function
- **global** keyword is used for creating global variables inside the block or function.

Python String :

- String = set of characters.
- It is written inside either double quotes or single quotes.
- You can use quotes inside quote
  Ex., print(“this is called ‘quality’.”)
  o/p : this is called ‘quality’.
- You can use multiline string like
  Ex.,
  ```
  a = """Lorem ipsum dolor sit amet,
  consectetur adipiscing elit,
  sed do eiusmod tempor incididunt
  ut labore et dolore magna aliqua."""
  print(a)
  ```
- In operator = check given value is present
  Ex., a = “I am so tired because of travelling. ”
  if “tired” in a:
  print(“yes it is present”)
  print(“tired” in a)
  o/p : yes it is present
  True
- not in operator = check given value is not present (same as “in” operator)

Decision making:

If statement :

Syntax:

```
if condition :
        Block of code
```

```
Num = 4
if num%2 == 0:
        print("number is even.")
```

Elif statement :

Syntax :

```
if condition :
        Block of code
elif condition :
        Block of code
```

```
Num = 100
If num > 99 :
        print("its greater than 99 ")
elif num < 99:
        print("its smaller than 99 ")
```

If - else statement :

Syntax:

```
if condition:
        Block of code
else:
        Block of code
```

```
if True :
        print("its if block ")
else:
        print("its else block ")
```

Nested statement :

Syntax:

```
if condition:
        if condition:
```

```
        Block of code
    else:
        Block of code
```

```
if  morning :
    if good == True :
        print("have a nice day ")
    else:
        print("have a bad day ")
```

- **LOOPs**
  - **while loop**

    Syntax :

```
while condition:
    Block of code
```

    Example :

```
i = 1
while i <= 10:
    print(i)
```

    - **break** keyword used for terminate the loop when you want , even condition is true

      Example:

```
i = 1
while i <= 10:
    print(i)
    if(i==6):
        break
```

    - **continue** keyword used to skip specific iteration and then again restart the loop

      Example:

```
i = 1
while i <= 10:
    print(i)
    if(i==6):
        continue
```

    - We can also use **else** with while loop

      Example :

```
i = 1
while i <= 10:
        print(i)
else:
        print("loop is ended after 10 ")
```

- **for loop**

Syntax:

```
for variable_name in range:
        Block of code
```

Example :

```
fruit = ["apple","banana","fig"]
for x in fruit :
        print(x)
```

- Looping through a string

  Example :

  ```
  for x in "banana":
          print(x)
  ```

- Using **break** keyword

  Example:

  ```
  for x in "banana":
          print(x)
          if(x=="n"):
                  break
  ```

- Using **continue** keyword

  Example:

  ```
  Alpha = ['A','B','C','D']
  for x in Alpha :
          print(x)
          if(x == 'C'):
                  continue
  ```

- **range()** function is used for sequence of numbers start from 0 (by default) and increment by 1 (by default)

  Example :

  ```
  for num in range(10):
          print(num)
  ```

  - **range(start , limit)**

    Example:

    ```
    for num in range(2,10):
    ```

```
        print(num)
```
o/p: 2,...,9

- **range(start,limit,iteration)**

  Example:
```
for i in range(2,13,2):
        print(i)
```
o/p: 2,4,6,8,...,12

➔ **Using else in for loop**

Example :
```
for a in “Sayma”:
        print(a)
else:
        print(“loop is finished …”)
```

➔ **Nested for loop**

Example:
```
for a in “sayma”:
        For b in “kazi”:
                print(a,b)
```

➔ **Using pass keyword (used for future changes or code in loop)**

Example :
```
for i in “banana”:
        Pass
```